

Backend CV AI Cyber Threat Prevention

Summary

- Built backend services for a Windows desktop cyber-defense app: database, monitoring, protection, encryption, and model serving.
- Focus on robustness: SQLite persistence, safe encryption for quarantined files, background services for monitoring and scanning.

Key Dependencies

- ``sqlite3`` (via ``DatabaseManager``): lightweight embedded DB storing logs, encrypted_files, settings.
- ``watchdog``: filesystem watcher for monitored folder events.
- ``cryptography`` (**Fernet**): symmetric encryption of quarantined files.
- ``scikit-learn`` + ``joblib``: model persistence and ML inference for phishing detection.
- ``psutil``: system telemetry (CPU, memory, network) for monitoring service.
- ``PyQt6`` event bus integration: backend emits events to frontend UI.

Architecture (concise)

- ``DatabaseManager`` (`database/db.py`): initializes tables and provides helper methods like ``log_phishing``, ``log_threat``, ``log_monitoring``, ``execute``, ``fetch_all``.
- **Services:**
 - ``MonitoringService`` (`monitoring/service.py`): samples system metrics, runs IsolationForest anomaly detection, watches ``MONITORED_FOLDER``, and uses ``MalwareScanner`` for new files.
 - ``ProtectionService`` (`protection/service.py`): quarantine files by copying to ``QUARANTINE_DIR``, encrypting using ``FileEncryptionManager``, and recording the entry in ``encrypted_files`` table.
 - ``PhishingService``/``PhishingModelManager``: exposes ``predict()`` and handles model loading, thresholding, and retraining.

Database schema (important tables)

- ``phishing_logs(id, input_text, prediction, confidence, source, created_at)``
- ``threat_logs(id, threat_type, severity, description, source, created_at)``
- ``monitoring_logs(id, cpu, memory, processes, network_sent, network_recv, threat_score, severity, anomaly_score, created_at)``
- ``encrypted_files(id, file_path, key_path, status, created_at)``
- ``protection_logs(id, action, target, result, created_at)``

How backend connects to frontend (short)

- ``CyberDefenseWindow`` constructs ``DatabaseManager``, ``AppEventBus``, and services, then passes them into UI components.
- UI components subscribe to ``AppEventBus`` signals (``alert``, ``log``, ``metrics_updated``) to update widgets.
- Example flow: ``MonitoringService`` detects malicious file â†’ calls ``ProtectionService.quarantine_file(path)`` â†’ encrypts and writes DB

row â†’ emits `event\_bus.emit\_alert(...)` â†’ GUI shows notification and can display quarantine entries by querying `DatabaseManager.fetch\_all("SELECT ... FROM encrypted\_files")`.

Short code excerpts

DatabaseManager usage (init + query)

```
python
initialize
db = DatabaseManager()
insert a protection log
db.log_protection('quarantine_file', str(source), 'encrypted and
quarantined')
fetch quarantine rows
rows = db.fetch_all('SELECT id, file_path, key_path, status,
created_at FROM encrypted_files ORDER BY id DESC LIMIT 100')
```

ProtectionService.quarantine_file (core steps)

```
python
QUARANTINE_DIR.mkdir(parents=True, exist_ok=True)
target = QUARANTINE_DIR / source.name
shutil.copy2(source, target)
encrypted_path = self.encryption.encrypt_file(target)
self.database.execute(
    "INSERT INTO encrypted_files (file_path, key_path, status,
created_at) VALUES (?, ?, ?, datetime('now'))",
    (str(source), str(self.encryption.key_path), "encrypted"),
)
```

Event flow to frontend (emit alert)

```
python
if self.event_bus:
    self.event_bus.emit_alert({
        "title": "File quarantined",
        "message": source.name,
        "severity": "high",
        "module": "protection",
    })
```